VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Machhe, Belagavi – 590018, Karnataka

2022 SCHEME – BCS714D

ASSIGNMENT 1 REPORT

ON

Distributed Food and Nutrient Analytics Using MongoDB, Hadoop MapReduce, Pig and Hive

Track B – Part 1: Three Core Themes

Submitted as part of the coursework for Big Data Analytics

Sl. No.	Student name	USN	Marks	Signature
1	Abhishek R	1JS23CS004		
2	Adithya V	1JS23CS005		
3	Dhanush Singh G	1JS23CS048		

Group No.: 18 | Section: CSE A

Course Faculty: Sandhiya B

Department of Computer Science and Engineering

JSS ACADEMY OF TECHNICAL EDUCATION

JSS Campus Road, Dr. Vishnuvardhan Road, Bengaluru – 560060

Academic Year: 2026–2027 (ODD Semester)

Contents

Section	Page
1. Introduction, Objectives and Dataset	3
2. Environment and System Architecture	4
3. MongoDB Ingestion and Aggregation	5
4. MongoDB Profiling and Analytical Results	6
5. HDFS File Management and MapReduce	7
6. Pig Transformation and Hive Tables	8
7. Hive Performance and Storage Results	9
8. Conclusion and References	10

Dataset: USDA FoodData Central, October 31, 2024 CSV release. Group 18, CSE A. The report presents the implementation and recorded measurements for the linked 50,000-food development dataset.

1. Introduction, Objectives and Dataset

This project analyses the USDA FoodData Central October 2024 CSV package using document processing and distributed batch analytics. Food descriptions, nutrient definitions and nutrient amounts are linked through identifiers, transformed into line-oriented records and processed across MongoDB, Hadoop MapReduce, Pig and Hive.

The objectives are to ingest and index source documents, calculate food and nutrient summaries, demonstrate distributed file operations, clean prepared records, and compare equivalent Hive queries over Text and RCFile storage. A Flask dashboard presents the recorded evidence as the Track B practical extension.

1.1 Dataset scope

Item	Full release	Analysed subset
Food records	2,063,876	50,000
Food–nutrient records	26,848,323	1,126,928
Nutrient definitions	477	477
Source CSV tables	34	3 used for linked analytics

The development subset selects the first 50,000 foods by source-row ordinal and includes all their linked nutrient records. It is a deterministic development selection, rather than a representative statistical sample. All 34 source CSV files were separately imported into the full-source MongoDB database.

1.2 Principal source tables

food.csv provides food identifiers, names and types; food_nutrient.csv links food identifiers to nutrient identifiers and amounts; nutrient.csv supplies nutrient names and units. The joins use fdc_id and nutrient_id. Prepared TSV records contain food ID, type, category, nutrient ID, nutrient name, unit and amount.

2. Environment and System Architecture

The reported experiment uses a three-container Hadoop cluster on one laptop: one master and two workers connected through a Docker network. The host runs Ubuntu 24.04 under Windows Subsystem for Linux 2, with approximately 10 GB allocated to WSL and four processors. This is a container development environment, distinct from three physical lab computers.

Component	Project configuration
Runtime	Java 8; Hadoop 2.10.2; Python host / container runtimes
Analytics	MongoDB 7.0.16; Pig 0.17.0; Hive 2.3.9
Master	HDFS NameNode and YARN ResourceManager; Pig/Hive clients
Workers	Two DataNodes and two NodeManagers
Storage	HDFS replication factor 2; persistent project directories

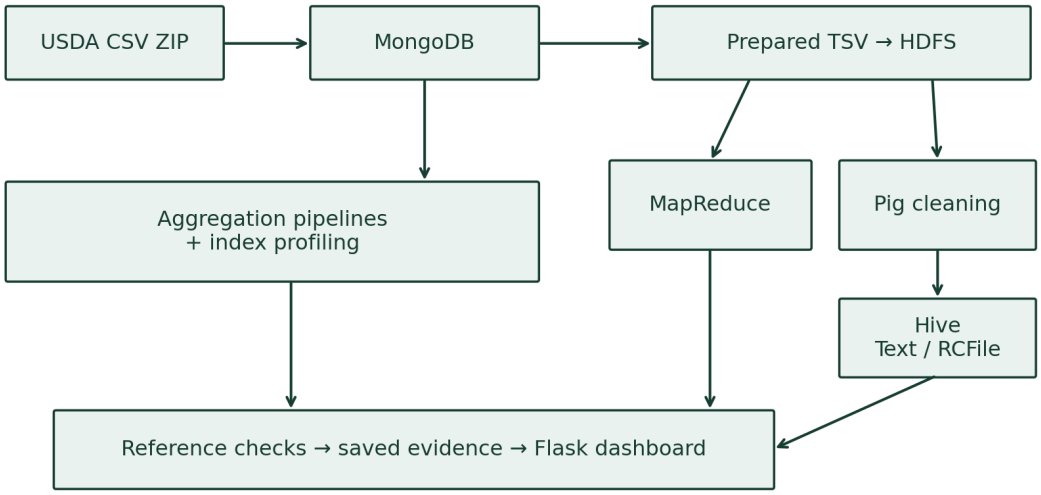


Figure 1. FoodData Central processing branches and evidence flow.

Cluster verification records two live DataNodes and two YARN workers. Prepared files are shared through HDFS; Python Streaming and Pig execute through YARN. MongoDB is run separately and switched with Hadoop during the laptop workflow to reduce concurrent memory demand.

3. MongoDB Ingestion and Aggregation

3.1 Collections and linkage

The analysed subset uses raw_food, raw_food_nutrient and raw_nutrient as source collections. The derived foods collection stores each food with its linked nutrient array, while metadata records source identity, row counts, selection and completion status. The original records remain unchanged when analytical outputs are generated.

A source example is food ID 1105904, “WESSON Vegetable Oil 1 GAL”. Its linked observations include polyunsaturated fatty acids (53.33 G), cholesterol (0 MG) and energy (867 KCAL). These are individual recorded amounts; totals across groups are separate analytical results.

3.2 Aggregation pipelines

Operator	Role in this implementation
$match	Select valid amounts and restrict groups or records.
$group	Calculate counts, sums and averages.
$project	Choose fields and present calculated answers.
$unwind	Expand the linked nutrient array into observations.
$facet	Produce food count, type distribution and nutrient coverage together.

The foods_by_type pipeline counts foods by data type. The nutrient_summary pipeline expands nutrient arrays and groups observations by food type and nutrient identifier. The overview pipeline combines multiple summaries in one faceted result. The displayed examples are limited previews, not limits on the actual analysis.

3.3 Indexes

Indexes on raw_food.fdc_id, raw_food_nutrient.fdc_id and raw_nutrient.id support food lookups and the food–nutrient joins. The profiling experiment forces an unindexed collection scan and an indexed lookup for the same food identifier.

4. MongoDB Profiling and Analytical Results

The same lookup was executed five times per plan in alternating order. The supplied 50,000-food dashboard evidence reports the following median execution statistics.

Plan	Docs checked	Index keys	Returned	Median ms
Collection scan	50,000	0	1	42
Indexed lookup	1	1	1	0

Both plans return one matching food. The index reduces documents examined from 50,000 to one. The displayed zero milliseconds is below the server timer’s millisecond resolution; it does not mean the lookup required no work. A finite speedup ratio is therefore not calculated from this zero value.

4.1 Selected food-type counts

Food type	Foods
branded_food	32,925
sr_legacy_food	7,793
sub_sample_food	5,967
market_acquisition	2,506
sample_food	756

These are the five displayed groups, not the complete type distribution. The faceted overview confirms 50,000 foods and approximately 22.5 linked nutrient observations per food.

4.2 Example nutrient interpretation

For branded_food protein (nutrient ID 1003), 32,840 usable observations have a total recorded amount of 242,187.06 G and a mean of approximately 7.37476 G per observation. Mean equals total divided by observation count. These figures summarise a product group, not the protein in one named food or an amount consumed.

Raw document previews were captured later from a metadata-matching database and are labelled accordingly in the dashboard. They demonstrate linkage without implying that the later preview capture was part of the original benchmark timing.

5. HDFS File Management and MapReduce

5.1 Distributed file management

Prepared records are uploaded under /user/root/fdc/realdev_1792244956898668695 in the laptop cluster. The prepared subdirectory contains TSV input; mapreduce stores reducer output, pig_clean stores transformed records and hive_rc holds the RCFile copy.

Operation	Demonstrated behaviour
mkdir -p	Create the run’s raw, prepared and temporary demo folders.
put	Upload input files and a small verification probe.
cat	Read the probe and inspect stored records.
get	Download the probe and verify identical bytes.
rm -r	Delete the temporary demo directory, retaining analytical data.

5.2 Python Streaming on YARN

The mapper reads each seven-field nutrient record, validates its numeric amount and excludes negative values. It emits a food-type/nutrient key with the amount and a count of one. Hadoop partitions and sorts the keys; reducers combine each group into observation count, total amount and mean amount without buffering the whole dataset.

For example, the branded_food / protein group produces 32,840 observations, a total of approximately 242,187.06 and a mean of 7.37476. The dashboard rounds some reducer values for display; reconciliation uses the saved numeric outputs with an explicit floating-point tolerance.

5.3 Verification

Recorded evidence identifies two DataNodes and YARN workers worker1 and worker2. A sample Pi MapReduce job checks the execution environment before the food analytics job. The Pi estimate is a cluster diagnostic, not a nutrition result. The saved food results are checked against independent reference calculations and corresponding MongoDB aggregates.

6. Pig Transformation and Hive Tables

6.1 Pig Latin cleaning

Pig reads the prepared HDFS nutrient records as a separate branch from MapReduce. It requires non-null food ID, nutrient ID, food type and amount, and keeps amounts greater than or equal to zero. Clean records are written back to HDFS for Hive ingestion.

Prepared nutrient rows	Negative rows excluded	Clean rows
1,126,928	0	1,126,928

Zero excluded negative values means that this subset's prepared input already satisfies the nonnegative filter; the cleaning script still executes and validates its output. Pig does not modify MongoDB documents or clean the MapReduce output.

6.2 Equivalent Hive datasets

Table	Stored data and format
nutrients_text	External table over Pig's tab-separated cleaned records.
nutrients_rcfile	The same rows copied into gzip-compressed RCFile storage.

Each table has seven columns: fdc_id, data_type, food_category_id, nutrient_id, nutrient_name, unit and amount. The RCFile table is populated using INSERT OVERWRITE ... SELECT * FROM nutrients_text. The two stored datasets are created once per processing run and reused by all query repetitions.

6.3 Analytical queries

Query 1 groups all nonnegative nutrient observations by food type, nutrient ID and unit, returning COUNT, SUM and AVG. Query 2 performs the same calculation for protein only, with nutrient_id = 1003 and unit G. Each question is executed against both tables. One warmup and one measured execution per query/table pair give eight executions, not eight distinct analytical questions.

7. Hive Performance and Storage Results

The benchmark excludes warmups from the reported query times. Each table/query pair has one measured execution. Elapsed time includes Beeline startup and no cache flush is performed; the measurements describe this development run.

Question	Text (s)	RCFile (s)
All-nutrient summary	66.24	64.22
Protein summary	64.91	67.35

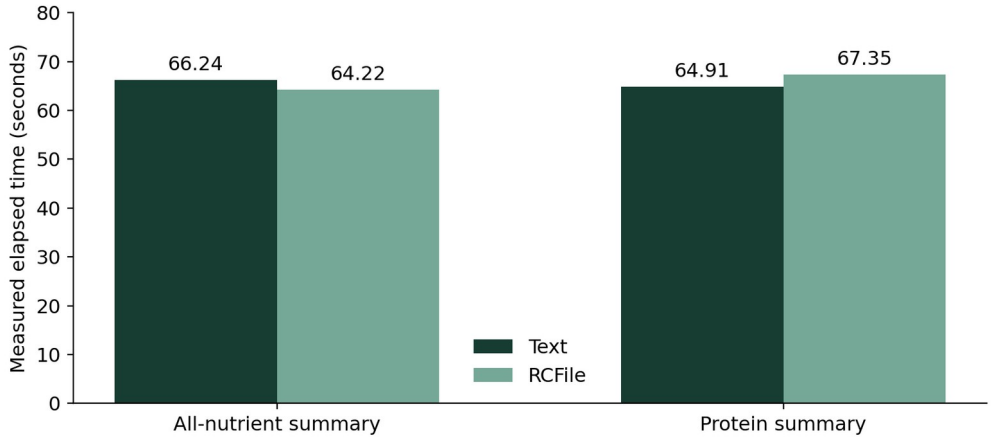


Figure 2. Identical query workloads measured on Text and RCFile.

Format	Logical bytes	Replicated bytes
Text	67,081,701	134,163,402
RCFile	6,622,922	13,245,844

RCFile uses approximately 90.13% less logical storage in this run. It is 2.02 seconds faster for the all-nutrient query and 2.44 seconds slower for protein. The result supports a substantial storage reduction, while these short timing measurements do not establish a consistent query-speed advantage. Both formats produce matching analytical answers.

8. Conclusion and References

The project implements all three core themes on a linked 50,000-food development dataset containing 1,126,928 nutrient observations. MongoDB ingestion and aggregation, HDFS file operations, YARN Streaming, distributed Pig transformation and Hive Text/RCFile queries form a repeatable, checked workflow.

The indexed lookup examines one document instead of 50,000. Pig retains all prepared observations for this selected subset, and equivalent Hive tables return matching analytical summaries. Gzip RCFile substantially reduces stored bytes, while the measured query times are similar and depend on the question. The Flask extension exposes the inputs, outputs and verification commands in an accessible interface.

References

1. USDA FoodData Central — Download Datasets. https://fdc.nal.usda.gov/download-datasets/

2. MongoDB — Aggregation Pipelines. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

3. Apache Hadoop — Cluster Setup. https://apache.github.io/hadoop/hadoop-project-dist/hadoop-common/ClusterSetup.html

4. Apache Pig 0.17 — Documentation. https://pig.apache.org/docs/r0.17.0/

5. Apache Hive — Language Manual. https://cwiki.apache.org/confluence/display/Hive/LanguageManual

6. Project source code. https://github.com/Dhanush-singh-g/BDA-Assignment